PI Calculation using Serial, MPI, and OpenMP

1. Problem Description

In this assignment, I implemented three different versions of a program to approximate the value of PI using numerical integration. The goal was to compare a serial implementation with parallel implementations using MPI and OpenMP.

The value of PI was approximated using the midpoint rectangle method with N = 100,000 iterations.

2. Approach

For the serial version, I used a simple loop that computes the sum of all iterations one at a time on a single processor. I used this version as the baseline for performance comparison.

For the MPI version, I split the total number of iterations across multiple processes using a cyclic distribution. Each process calculates a local sum, and then all local sums are combined using MPI_Reduce to get the final result.

For the OpenMP version, I used a parallel for loop with a reduction clause. This allowed multiple threads to compute parts of the sum at the same time while safely combining the results at the end.

3. Results

Compilation

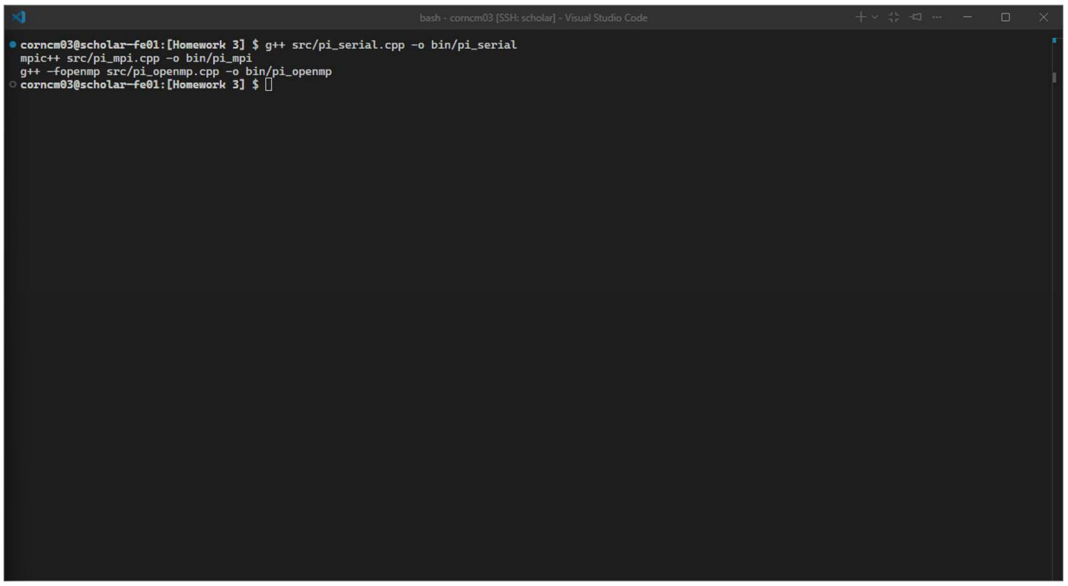


Figure: Compile Commands Completed

Serial Execution

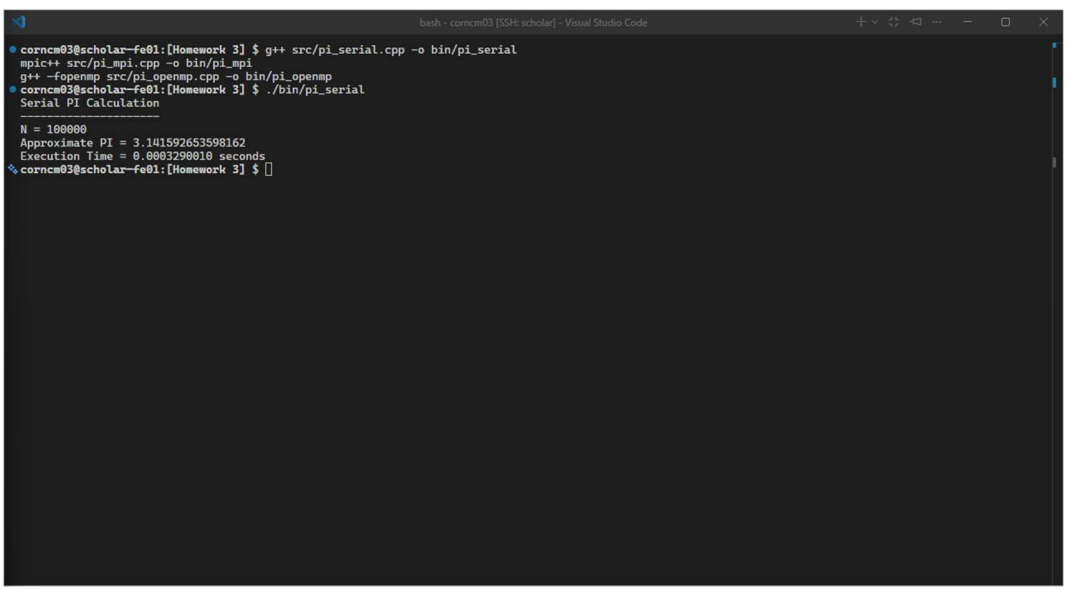


Figure: Pi Serial Run

MPI Execution

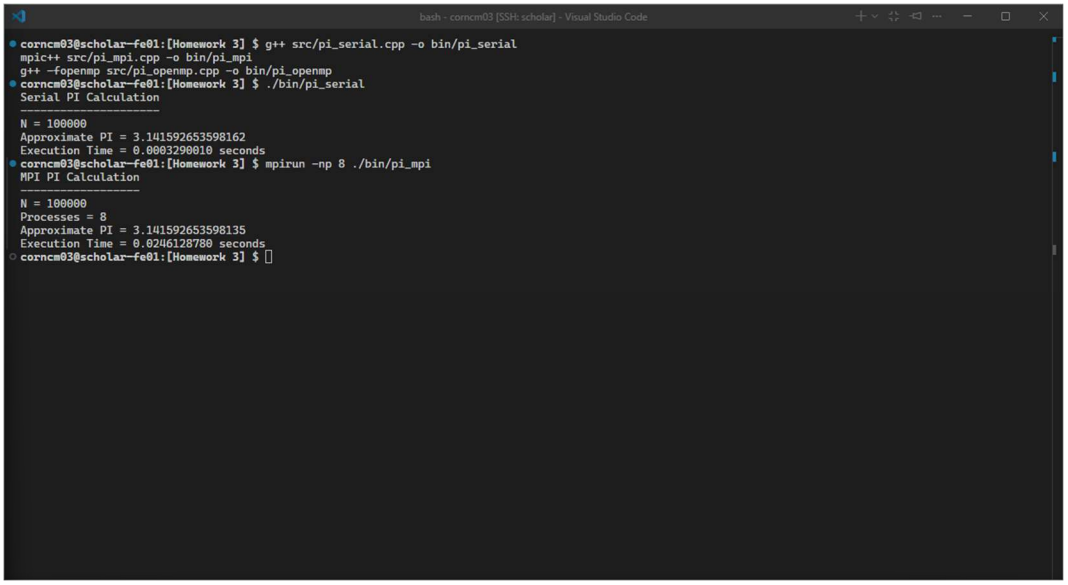


Figure: Pi MPI 8 Run

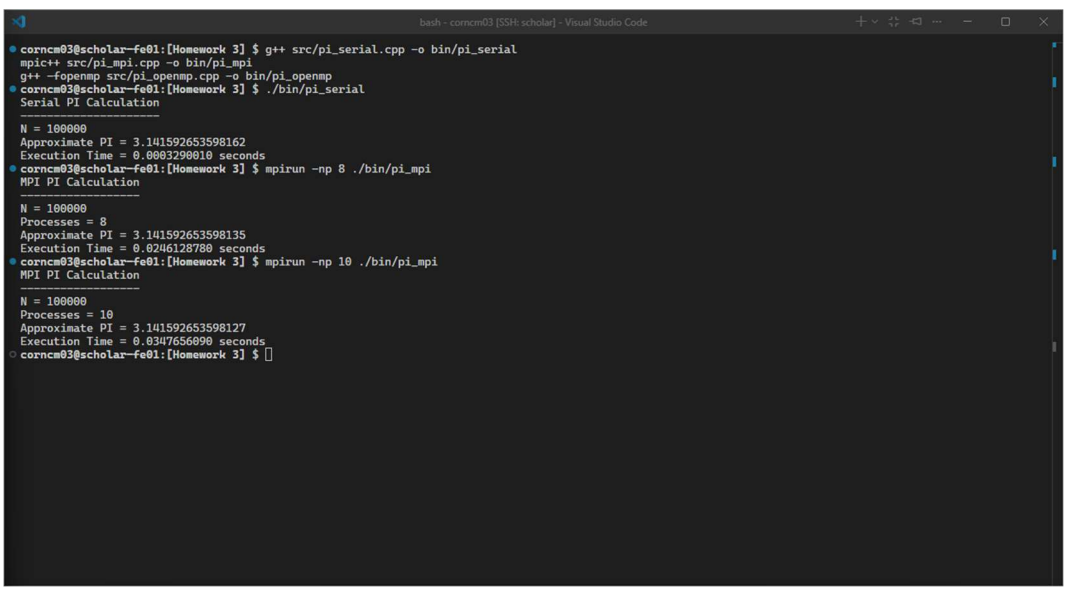
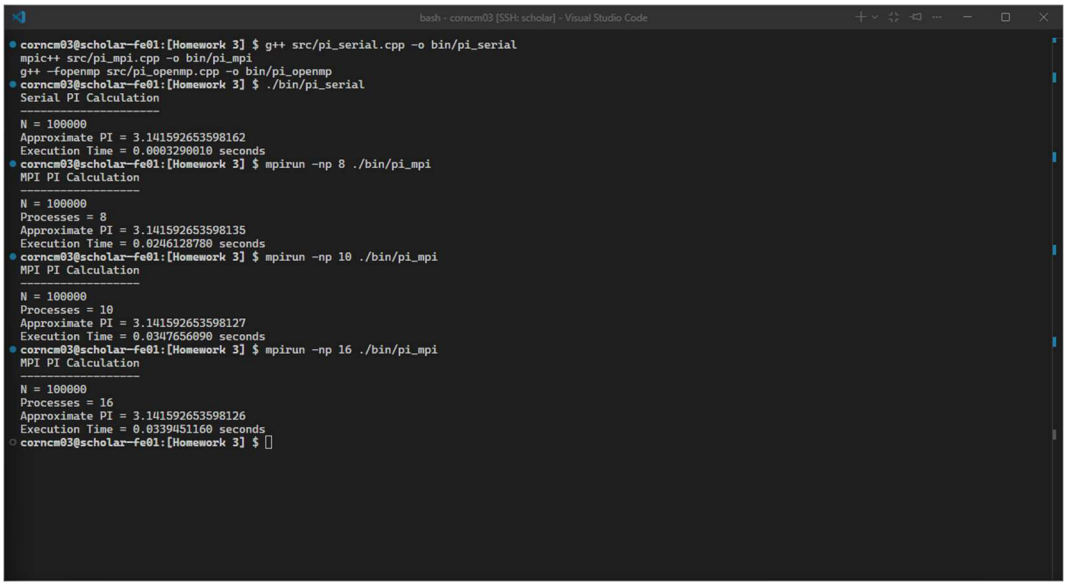
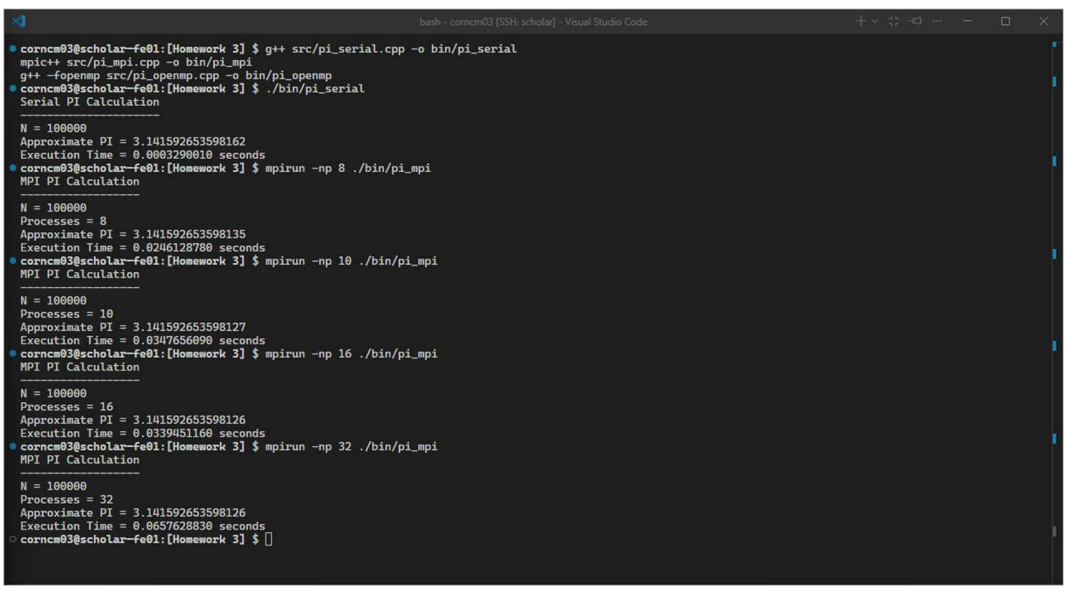


Figure: Pi MPI 10 Run



```
bash - corncm03 [SSH: scholar] - Visual Studio Code
corncm03@scholar-fe01:[Homework 3] $ g++ src/pi_serial.cpp -o bin/pi_serial
mpic++ src/pi_mpi.cpp -o bin/pi_mpi
g++ -fopenmp src/pi_openmp.cpp -o bin/pi_openmp
corncm03@scholar-fe01:[Homework 3] $ ./bin/pi_serial
Serial PI Calculation
---------------------
N = 100000
Approximate PI = 3.141592653598162
Execution Time = 0.0003290010 seconds
corncm03@scholar-fe01:[Homework 3] $ mpirun -np 8 ./bin/pi_mpi
MPI PI Calculation
------------------
N = 100000
Processes = 8
Approximate PI = 3.141592653598135
Execution Time = 0.0246128780 seconds
corncm03@scholar-fe01:[Homework 3] $ mpirun -np 10 ./bin/pi_mpi
MPI PI Calculation
------------------
N = 100000
Processes = 10
Approximate PI = 3.141592653598127
Execution Time = 0.0347656090 seconds
corncm03@scholar-fe01:[Homework 3] $ mpirun -np 16 ./bin/pi_mpi
MPI PI Calculation
------------------
N = 100000
Processes = 16
Approximate PI = 3.141592653598126
Execution Time = 0.0339451160 seconds
corncm03@scholar-fe01:[Homework 3] $
```

Figure: Pi MPI 16 Run



```
bash - corncm03 [SSH: scholar] - Visual Studio Code
corncm03@scholar-fe01:[Homework 3] $ g++ src/pi_serial.cpp -o bin/pi_serial
mpic++ src/pi_mpi.cpp -o bin/pi_mpi
g++ -fopenmp src/pi_openmp.cpp -o bin/pi_openmp
corncm03@scholar-fe01:[Homework 3] $ ./bin/pi_serial
Serial PI Calculation
---------------------
N = 100000
Approximate PI = 3.141592653598162
Execution Time = 0.0003290010 seconds
corncm03@scholar-fe01:[Homework 3] $ mpirun -np 8 ./bin/pi_mpi
MPI PI Calculation
------------------
N = 100000
Processes = 8
Approximate PI = 3.141592653598135
Execution Time = 0.0246128780 seconds
corncm03@scholar-fe01:[Homework 3] $ mpirun -np 10 ./bin/pi_mpi
MPI PI Calculation
------------------
N = 100000
Processes = 10
Approximate PI = 3.141592653598127
Execution Time = 0.0347656090 seconds
corncm03@scholar-fe01:[Homework 3] $ mpirun -np 16 ./bin/pi_mpi
MPI PI Calculation
------------------
N = 100000
Processes = 16
Approximate PI = 3.141592653598126
Execution Time = 0.0339451160 seconds
corncm03@scholar-fe01:[Homework 3] $ mpirun -np 32 ./bin/pi_mpi
MPI PI Calculation
------------------
N = 100000
Processes = 32
Approximate PI = 3.141592653598126
Execution Time = 0.0657628830 seconds
corncm03@scholar-fe01:[Homework 3] $
```

Figure: Pi MPI 32 Run

OpenMP Execution

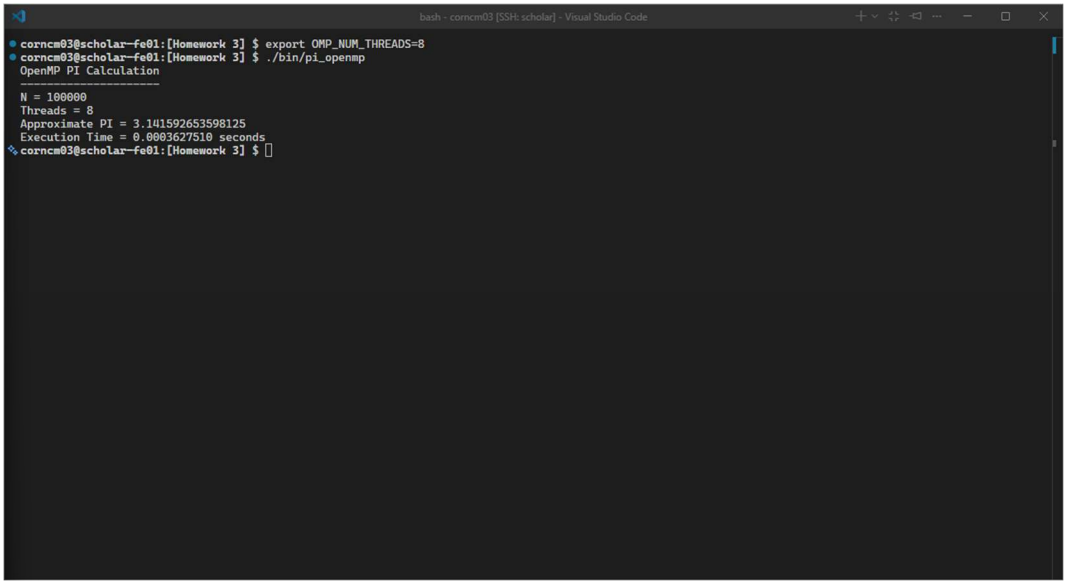
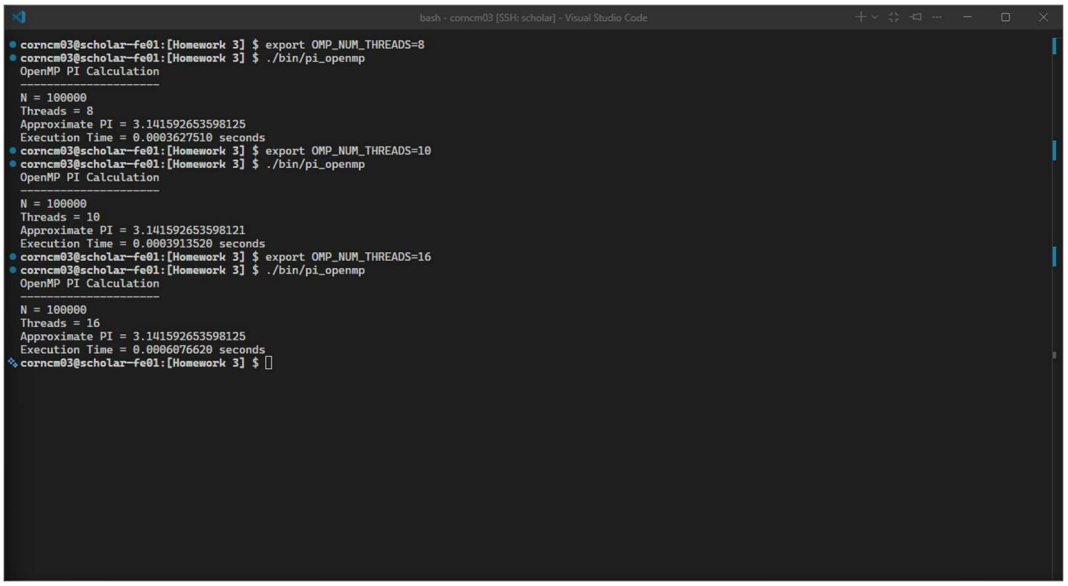


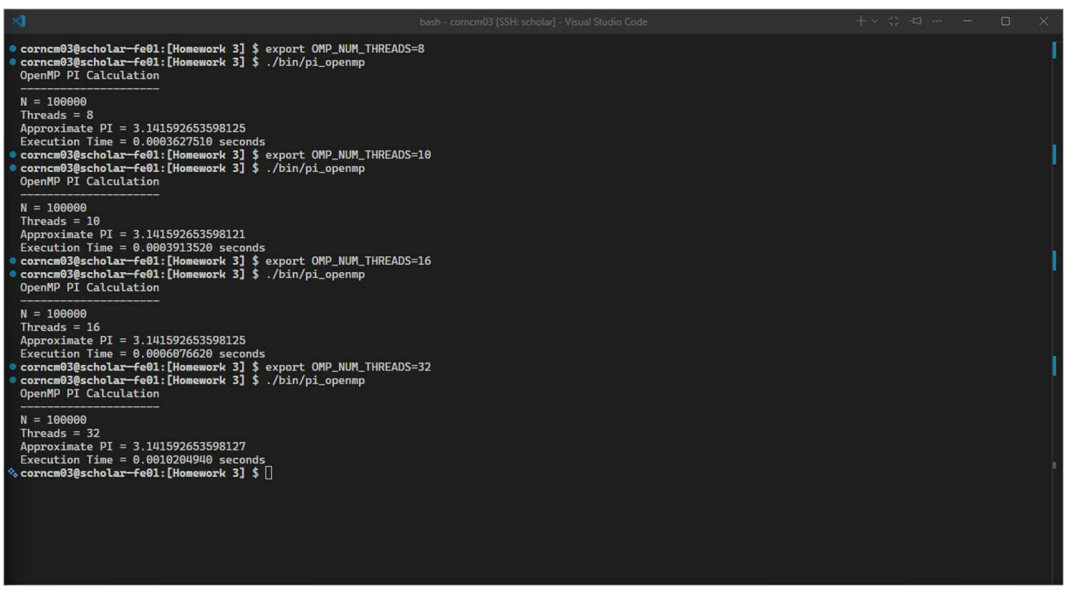
Figure: Pi OpenMP 8 Run

Figure: Pi OpenMP 10 Run



```
bash - corncm03 [SSH: scholar] - Visual Studio Code
corncm03@scholar-fe01:[Homework 3] $ export OMP_NUM_THREADS=8
corncm03@scholar-fe01:[Homework 3] $ ./bin/pi_openmp
OpenMP PI Calculation
---------------------
N = 100000
Threads = 8
Approximate PI = 3.141592653598125
Execution Time = 0.0003627510 seconds
corncm03@scholar-fe01:[Homework 3] $ export OMP_NUM_THREADS=10
corncm03@scholar-fe01:[Homework 3] $ ./bin/pi_openmp
OpenMP PI Calculation
---------------------
N = 100000
Threads = 10
Approximate PI = 3.141592653598121
Execution Time = 0.0003913520 seconds
corncm03@scholar-fe01:[Homework 3] $ export OMP_NUM_THREADS=16
corncm03@scholar-fe01:[Homework 3] $ ./bin/pi_openmp
OpenMP PI Calculation
---------------------
N = 100000
Threads = 16
Approximate PI = 3.141592653598125
Execution Time = 0.0006076620 seconds
corncm03@scholar-fe01:[Homework 3] $
```

Figure: Pi OpenMP 16 Run



```
bash - corncm03 [SSH: scholar] - Visual Studio Code
corncm03@scholar-fe01:[Homework 3] $ export OMP_NUM_THREADS=8
corncm03@scholar-fe01:[Homework 3] $ ./bin/pi_openmp
OpenMP PI Calculation
---------------------
N = 100000
Threads = 8
Approximate PI = 3.141592653598125
Execution Time = 0.0003627510 seconds
corncm03@scholar-fe01:[Homework 3] $ export OMP_NUM_THREADS=10
corncm03@scholar-fe01:[Homework 3] $ ./bin/pi_openmp
OpenMP PI Calculation
---------------------
N = 100000
Threads = 10
Approximate PI = 3.141592653598121
Execution Time = 0.0003913520 seconds
corncm03@scholar-fe01:[Homework 3] $ export OMP_NUM_THREADS=16
corncm03@scholar-fe01:[Homework 3] $ ./bin/pi_openmp
OpenMP PI Calculation
---------------------
N = 100000
Threads = 16
Approximate PI = 3.141592653598125
Execution Time = 0.0006076620 seconds
corncm03@scholar-fe01:[Homework 3] $ export OMP_NUM_THREADS=32
corncm03@scholar-fe01:[Homework 3] $ ./bin/pi_openmp
OpenMP PI Calculation
---------------------
N = 100000
Threads = 32
Approximate PI = 3.141592653598127
Execution Time = 0.0010204940 seconds
corncm03@scholar-fe01:[Homework 3] $
```

Figure: Pi OpenMP 32 Run

4. Timing and Speedup Results

Version	Processes/Threads	Time (seconds)	Speedup
Serial	1	0.0003290010	1.0
MPI	8	0.0246128780	0.0134
MPI	10	0.0347656090	0.0095
MPI	16	0.0339451160	0.0097
MPI	32	0.0657628830	0.0050
OpenMP	8	0.0003482320	0.9448
OpenMP	10	0.0003913520	0.8407
OpenMP	16	0.0006076620	0.5414
OpenMP	32	0.0010204940	0.3224

5. Discussion

I expected the parallel versions to perform better than the serial version, but the results showed that the serial version was actually faster for this problem size.

I think this happened because N = 100,000 is relatively small, so the actual computation finishes very quickly. The overhead of creating MPI processes and OpenMP threads ends up taking more time than the computation itself.

For MPI, performance got worse as I increased the number of processes. This makes sense because more processes means more communication and synchronization overhead, especially when the workload per process becomes smaller.

OpenMP performed better than MPI, but it still did not outperform the serial version.

This helped me understand that shared-memory parallelism has less overhead than distributed-memory parallelism, but still has some cost when creating and managing threads.

Overall, this assignment showed me that parallel programming is not always faster, especially when the problem size is too small to benefit from it. If N was much larger, I would expect the parallel versions to start outperforming the serial version because the computation would begin to outweigh the overhead.

6. Conclusion

From this assignment, I learned how to implement parallel programs using both MPI and OpenMP, and how to measure and compare their performance.

I also learned that overhead plays a big role in parallel computing, and that increasing the number of processes or threads does not always lead to better performance.